

InStock: an Economic Sector Impact and Financial Sentiment Analysis: A Consensus-Based Ensemble Model Approach

Kelvin Leandi
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
kelvin.leandi@binus.ac.id

Gavinn Aloys
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
gavinn.aloys@binus.ac.id

Nathanael Setiorahardjo
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
nathanael.setiorahardjo@binus.ac.id

Mohammad Faisal Rifiarrasyid,
S.Kom., M.Kom.
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
mohammad.rifiarrasyid001@binus.ac.id

Abstract—Analyzing public sentiment in financial news is critical for predicting market volatility and economic sector impacts. While deep learning transformer models currently dominate natural language processing, applying them to localized, non-English markets presents significant linguistic and computational challenges. This study engineers a real-time web application to extract sentiment and sector-level intelligence from Indonesian financial news using a five model parallel architecture. The system integrates a traditional Support Vector Machine, a Bidirectional Long Short Term Memory network, DistilBERT, RoBERTa, and the multilingual XLM RoBERTa, aggregating their probabilistic predictions through a soft voting consensus mechanism. Empirical evaluation on a manually annotated corpus of Indonesian business articles revealed that a domain-pretrained transformer, DistilBERT fine-tuned on Indonesian text, achieved the strongest individual performance, reaching a peak Matthews Correlation Coefficient of 0.939, narrowly ahead of the uniformly-weighted soft-voting ensemble (MCC 0.934) and the TF-IDF-based Support Vector Machine baseline (MCC 0.912). These findings indicate that while parallel cloud-based transformer pipelines and traditional statistical baselines are both highly viable for Indonesian financial text categorization, an equally-weighted ensemble does not automatically surpass its strongest constituent model, motivating the adaptive weighting strategies identified as future work.

Keywords—Natural Language Processing, Financial Sentiment Analysis, Ensemble Learning, Support Vector Machine, Indonesian Stock Market, Text Categorization

I. INTRODUCTION

The sheer volume of financial news published daily significantly impacts market volatility and various economic sectors. Manually processing this vast amount of unstructured information to extract actionable economic

sentiment is computationally unfeasible, creating a critical need for automated Natural Language Processing (NLP) techniques to discover untapped potential in financial markets [1]. Historical research has consistently demonstrated that public mood and sentiment embedded in textual data can serve as strong predictive indicators for stock market fluctuations [14].

While general NLP models and traditional financial dictionaries [13] exist, they often fail to capture the highly specific nuances of the economic lexicon, leading to inaccurate market predictions. This limitation has necessitated a paradigm shift from traditional lexicons toward deep learning and domain-specific pre-trained language models [3]. Architectures such as FinBERT have proven demonstrably superior to general models in understanding complex economic contexts [2]. However, applying English-centric models directly to the Indonesian financial market introduces significant linguistic bottlenecks. Capturing the localized context and specific jargon of the Jakarta Composite Index (IHSG) requires robust cross-lingual capabilities or models specifically benchmarked for Indonesian NLP [15] to ensure that regional economic slang is accurately interpreted.

Furthermore, simply predicting whether a news article holds a general positive or negative sentiment is insufficient for robust, real-world application. To provide actionable intelligence to investors, it is equally critical to perform sector-level sentiment analysis to identify exactly which economic domains, such as Technology, Finance, or Energy & Mining, which are directly affected by these textual signals [4].

Finally, relying on a single computational architecture can introduce specific mathematical biases or architectural limitations. Traditional statistical methods like Support

Vector Machines (SVM) continue to provide a strong, high-dimensional baseline for text categorization [7], while deep neural networks such as Bidirectional LSTMs successfully capture sequential word relations [8]. More recently, the introduction of the attention mechanism [12] has driven the development of state-of-the-art bidirectional transformers like BERT [11], its disentangled attention variations [9], and highly efficient distilled counterparts like DistilBERT [10]. Extensive research demonstrates that integrating these diverse architectural approaches into a unified ensemble model significantly improves financial aspect and sentiment predictions [5]. By leveraging a soft-voting consensus mechanism over these transformer and deep learning outputs, systems can achieve higher probabilistic reliability than standalone classification techniques [6].

Therefore, the objective of this project is to develop a highly optimized, consensus-based NLP web application capable of predicting the sentiment of Indonesian financial news and simultaneously identifying the specific economic sectors affected. By engineering and deploying an ONNX-optimized ensemble architecture comprising SVM, BiLSTM, DistilBERT, RoBERTa, and the multilingual XLM-RoBERTa, this system aims to deliver sub-second, domain-specific sentiment classifications that overcome the limitations of individual standalone models.

The live deployment of the ensemble architecture can be accessed at <https://instockskyg.vercel.app/>, and the complete codebase is publicly available at <https://github.com/KAELGOGO/KYG-NLP-PERBANDIN-GAN-5-MODEL-AI-NEWS-SENTIMEN>.

II. RELATED WORKS

The evolution of financial sentiment analysis has transitioned from basic statistical counting to highly contextual, deep-learning-driven architectures. This section reviews the foundational methodologies and recent advancements that inform the proposed ensemble approach.

A. From Lexicons to Transformer Models

Early approaches to financial sentiment analysis relied heavily on dictionary-based methods. Loughran and McDonald [13] demonstrated that general-purpose lexicons often misclassify financial text, necessitating domain-specific financial dictionaries to accurately capture corporate liability and market sentiment. Similarly, early social media sentiment studies, such as those by Bollen et al. [14], proved that public mood signals could successfully predict stock market movements. However, these bag-of-words approaches struggle with syntactic context.

To resolve this, the field shifted toward deep learning and, subsequently, the attention mechanism introduced by Vaswani et al. [12]. This led to the dominance of Bidirectional Encoder Representations from Transformers (BERT) [11], which revolutionized context-heavy text classification. Mishev et al. [3] conducted extensive evaluations proving that transformer-based models significantly outperform traditional lexicon-based approaches in financial NLP. This was further specialized by

Araci [2] with the development of FinBERT, a pre-trained model fine-tuned explicitly on financial corpora to natively understand economic jargon.

B. Efficiency and Multilingual Adaptations

As models grew in complexity, deployment latency became a critical engineering constraint. Sanh et al. [10] addressed this with DistilBERT, which retains 97% of standard BERT's language understanding capabilities while operating 60% faster, making it highly viable for real-time market scanning. Parallel advancements in attention mechanisms, such as DeBERTa [9], improved syntactic dependency calculations, though often at the cost of inference speed.

Furthermore, while models like FinBERT excel in English, they exhibit severe performance degradation when applied to localized markets like Indonesia. Research by Koto et al. [15] on IndoLEM and IndoBERT highlighted the critical necessity for language-specific benchmarking and multilingual pre-training (such as XLM-RoBERTa) to accurately map domestic economic sentiment.

C. Ensemble Architectures and Sector-Level Impact

A recurring limitation in single-model deployments is architectural bias. To mitigate this, Piao and Breslin [5] demonstrated that integrating diverse deep neural networks into an ensemble approach significantly enhances the accuracy of financial aspect predictions. Alissa and Alzoubi [6] further validated this by employing a majority-voting consensus mechanism across multiple transformers, proving that aggregated probabilistic outputs yield higher reliability than standalone classifications.

Beyond binary sentiment, actionable market intelligence requires entity-specific mapping. Almalis et al. [4] emphasized the necessity of sector-level sentiment analysis, utilizing deep learning to attribute sentiment directly to specific economic domains (e.g., Finance, Technology). Shahapur et al. [1] recently reinforced this, showing that combining NLP-driven sentiment with targeted sector identification is the key to discovering untapped, actionable potential in financial markets.

The proposed architecture in this study builds directly upon these foundations by unifying traditional statistical baselines [7], sequential memory networks [8], and high-speed transformers [10] into a soft-voting ensemble, optimized specifically for the Indonesian financial sector.

III. METHODOLOGY

The proposed system implements an end-to-end Natural Language Processing (NLP) pipeline designed to ingest Indonesian financial news, evaluate sentiment polarity via a parallel model matrix, and extract targeted economic sector impacts. The architecture is systematically divided into five functional phases: Data Ingestion and Preprocessing, Vectorization and Encoding, Baseline Architecture Construction, Consensus Modeling, and Aspect-Based Sector Extraction.

A. Dataset Origin and Labeling Strategy

The primary static corpus utilized for system training and validation was sourced via an automated data ingestion pipeline that aggregated over 200,000 raw financial headlines (2018–2025) from CNBC Indonesia and Bisnis.com, executed using Python's undetected-chromedriver to circumvent publisher anti-bot mechanisms.

To eliminate subjective annotation bias and establish a rigorous ground truth, an initial subset of 150 articles, exactly 50 Positive, 50 Negative, and 50 Neutral, was manually annotated in direct consultation with a financial domain expert to establish classification heuristics. Using the patterns established in this expert-calibrated baseline, the research team manually labeled the remaining text to curate a final corpus of 3,300 headlines, perfectly balanced at 1,100 articles per class, eliminating the Neutral-class skew typically observed in financial journalism corpora.

B. Text Preprocessing and Tokenization Pipeline

Prior to model ingestion, raw unstructured text payloads undergo a deterministic multi-stage serialization pipeline:

- **Noise Reduction:** Regular expression (Regex) filters remove structural HTML fragments, URLs, alphanumeric noise, and non-ASCII characters, while explicitly preserving key financial indicators (e.g., "\$", "%", symbols indicating upward or downward trajectories like "↑" or "↓").
- **Text Standardization:** Text strings are converted to lowercase, and structural contractions are normalized.
- **Dual-Path Tokenization:** Due to the distinct architectural requirements of traditional classifiers versus transformer backbones, tokenization splits into two separate execution lines:
 - **Path A (Statistical Baseline):** Implements word-level tokenization, followed by Term Frequency-Inverse Document Frequency (TF-IDF) feature mapping to convert clean sequences into sparse matrices. To optimize the extraction of highly formulaic economic terminology, the vectorizer was parameterized to restrict the vocabulary. The vector space was strictly bounded to the top 5,000 most statistically significant text features (max_features=5000) to prevent high-dimensional overfitting before the sparse matrices were routed to the linear classifier.
 - **Path B (Transformer Architectures):** Payloads pass directly to sub-word tokenization networks (WordPiece and Byte-Pair Encoding) native to each respective transformer's architecture via the Hugging Face engine.

- **Class Balance Safeguard:** Prior to model fitting, SMOTE is applied to the vectorized training partition for the SVM, and RandomOverSampler is applied to the raw text training partition for the transformer models, as a defensive measure against partition-induced class imbalance. Given the corpus's balanced composition and the stratified split, both steps produced no additional synthetic samples in this run.

C. Deployed Model Matrix

To achieve predictive robustness, the inference layer runs a diverse 5-model parallel matrix, tracking historical advancements in NLP engineering:

- **Support Vector Machine (SVM):** Serves as the statistical machine learning floor, projecting sparse TF-IDF vectors into a high-dimensional feature space using an optimized linear kernel.
- **Bidirectional Long Short-Term Memory (BiLSTM):** Establishes a deep learning baseline, processing textual inputs sequentially in both forward and backward directions to preserve long-range contextual token dependencies.
- **DistilBERT:** A compressed transformer initialized via knowledge distillation, retaining 97% of standard BERT contextual capabilities while minimizing edge-deployment latency.
- **RoBERTa:** An optimized variant of BERT trained on expanded datasets with dynamic masking patterns, capturing subtle semantic interactions within standard text.
- **XLM-RoBERTa:** A cross-lingual powerhouse pre-trained on massive multi-language corpora, providing zero-shot cross-lingual transfer efficiency and deep contextual representation of localized Indonesian financial nomenclature.

D. Consensus Mechanism (Soft-Voting Ensemble)

Rather than executing a crude hard-voting majority rule (which strips the models of their probabilistic nuances), the system aggregates predictions via a mathematically calibrated Soft-Voting Classifier.

During runtime inference, a target sequence x passes through all (M) models ($M = 5$) simultaneously. Each network generates a continuous probability distribution across the target sentiment classes. The consensus engine computes the arithmetic mean of these probability vectors. Formally, the consensus probability ($P(c|x)$) for a given sentiment class (c) is defined in equation 1:

$$P(c | x) = \frac{1}{M} \sum_{m=1}^M P_m(c|x) \quad (1)$$

where $(P_m(c|x))$ represents the confidence score generated by model (m) for class (c). The class that secures the highest averaged probability is crowned as the final consensus output. This mathematical aggregation ensures that high-confidence predictions from specialized transformers are weighed properly against low-confidence variance in baseline models.

E. Aspect-Level Information Extraction

Following the resolution of the consensus sentiment, the text payload is routed to an information extraction subsystem. The system implements a rule-based dictionary matching approach rather than heavy dependency parsing to ensure sub-second latency. The lexicon is a manually curated dictionary spanning five economic sectors, Finance, Energy & Mining, Consumer Goods, Technology, and Infrastructure, comprising 42 keywords and corporate entities in total (e.g., 'bca', 'suku bunga', and 'ojk' mapped to Finance; 'adaro' and 'tambang' mapped to Energy & Mining). Text payloads are normalized and sequentially cross-referenced against this lexicon in fixed sector order; the first sector with a matching keyword is assigned as the article's sector label, and articles matching no keyword default to a "General Market" category. This single-label, first-match design prioritizes deterministic sub-second latency over exhaustive multi-sector tagging. A limitation of the current substring-based matching is that short keywords (e.g., "bi", "ai") may occasionally match as fragments within unrelated words, a constraint future iterations could address through word-boundary-aware tokenization.

F. Evaluation Metrics and Empirical Audit Framework

To rigorously evaluate the consensus engine and audit how architectural imbalances affect the soft-voting ecosystem, standard accuracy is discarded as a primary benchmark. The system is mathematically analyzed using metrics that track multi-class correlation alignment, probabilistic calibration distortion, and deployment latency constraints.

- **Matthews Correlation Coefficient (MCC):** To evaluate multi-class robustness independently of class distributions, MCC is used as the primary metric. It calculates the correlation between actual and predicted outcomes on a scale of -1 (total disagreement) to +1 (perfect classification), preventing the model from being artificially rewarded for class-distribution artifacts rather than genuine predictive signal.
- **Logarithmic Loss (Log-Loss):** Because the consensus layer relies on a soft-voting classifier that processes continuous probability vectors rather than hard-label tokens, the quality of those probabilities must be audited. Multinomial Logarithmic Loss is utilized to measure the exact cross-entropy penalty between the ensemble's predicted confidence distributions and the true target representations. Minimizing Log-Loss ensures that the ensemble

output is not only accurate but mathematically well-calibrated for high-stakes financial risk management.

- **Operational Success Criteria and Comparative Benchmarks:** To evaluate the absolute real-world performance trade-offs under operational conditions, success is structurally audited against three boundaries:
 - **Baseline Parity:** The consensus-based ensemble must be comparatively benchmarked against a standalone traditional linear model (SVM) and a sequential neural network (BiLSTM) to measure the net scientific returns of transformer integration.
 - **Calibration Vulnerability:** The engine evaluates whether a uniform soft-voting configuration can maintain a Log-Loss ceiling of < 0.40 when subjected to architectural noise introduced by constituent networks.
 - **Inference Latency Boundary:** To ensure production readiness, the parallel execution line across all active models must maintain a stable end-to-end processing throughput of under 2.5 seconds per query on runtime production workers.

IV. IMPLEMENTATION AND RESULTS

To evaluate the efficacy of the proposed soft-voting consensus mechanism, the five constituent models and the final ensemble were subjected to a rigorous empirical audit. The evaluation utilized an 80-20 data split for training and testing, prioritizing implementation simplicity over a dedicated validation partition. Class distribution across the training and test sets was confirmed to be balanced (880 articles per class in training, 220 per class in testing). The pipeline retains SMOTE and RandomOverSampler as defensive safeguards against partition-induced imbalance; because the corpus's balanced composition meant no class fell below the majority count, neither step introduced synthetic samples, so the reported metrics reflect training directly on the original, unaugmented headlines.

A. Individual Model Performance and Baselines

The empirical results for the independent architectures are summarized in Table I and Fig. 1. DistilBERT_Indo achieved the strongest individual performance, reaching a Matthews Correlation Coefficient (MCC) of 0.939 and an Accuracy of 0.959, confirming that a transformer pretrained on Indonesian-language corpora can effectively capture the nuanced lexical structure of Indonesian financial journalism even under a constrained fine-tuning corpus.

The Support Vector Machine (SVM) baseline remained highly competitive, achieving an MCC of 0.912 and an Accuracy of 0.941, confirming that TF-IDF vectorization continues to provide a strong, computationally inexpensive floor for highly formulaic financial text. The BiLSTM

achieved an MCC of 0.862, while the remaining transformer variants, RoBERTa_Indo and XLM_RoBERTa_Multi, yielded MCCs of 0.830 and 0.815 respectively, trailing both DistilBERT_Indo and the SVM baseline. This gap suggests that checkpoint provenance (language-specific pretraining vs. a general multilingual checkpoint) had a larger effect on individual performance than architecture family alone.

TABLE I. COMPARATIVE MODEL PERFORMANCE ON TEST SET (N=660)

All Model Metric Comparison					
Model	Acc.	Weighted F1	Macro F1	MCC	Log-Loss
SVM	0.941	0.941	0.941	0.912	0.217
BiLSTM	0.908	0.908	0.908	0.862	0.322
DistilBERT_Indo	0.959	0.959	0.959	0.939	0.18
RoBERTa_Indo	0.886	0.886	0.886	0.83	0.347
XLM_RoBERTa_Multi	0.876	0.874	0.874	0.815	0.377
Ensemble (Soft-Voting)	0.956	0.956	0.956	0.934	0.175

a. Acc. stand for Accuracy

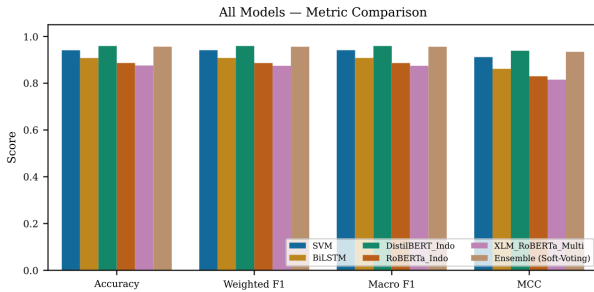


Fig. 1. Comparative Model Performance on Test Set (n=660)

B. Consensus Ensemble Performance and Analysis

The primary scientific objective of this study was to determine whether aggregating diverse architectures via a soft-voting mechanism could yield superior predictive reliability over the best individual baseline model. By averaging the continuous probability distributions of the constituent models, the ensemble achieved a peak Accuracy of 0.956, a Weighted F1-Score of 0.956, and an MCC of 0.934, narrowly trailing DistilBERT_Indo's standalone performance (MCC 0.939) while outperforming every other individual constituent, including the SVM baseline.

This result indicates that the uniform, equally-weighted soft-voting scheme, while effective at suppressing the variance of weaker constituents (RoBERTa_Indo and XLM_RoBERTa_Multi), is ultimately constrained by granting these weaker models equal influence alongside the strongest individual architecture. Because the ensemble blends five probability distributions of unequal quality at a fixed 1/5 weight each, it cannot mathematically exceed a sufficiently dominant single model. This finding directly motivates the dynamically adaptive weighting strategy identified as future work in Section V, a validation-informed

weighting scheme could plausibly allow the ensemble to close this gap.

To confirm whether these point-estimate differences reflect genuine architectural advantages rather than sampling variance, pairwise McNemar's tests were conducted on the test-set predictions. The difference between DistilBERT_Indo and the ensemble was not statistically significant ($\chi^2=0.062$, $p=0.803$), nor was the difference between SVM and the ensemble ($\chi^2=3.115$, $p=0.078$). The only statistically significant pairwise difference observed was between SVM and DistilBERT_Indo ($\chi^2=4.033$, $p=0.045$), indicating that while DistilBERT_Indo's advantage over the traditional baseline is robust, its advantage over the soft-voting ensemble specifically should be interpreted as a near-tie rather than a definitive ranking at this test-set size (n=660). Fig. 2. presents the complete per-class precision, recall, and F1-score breakdown across all six models, discussed in detail in Section IV.C.

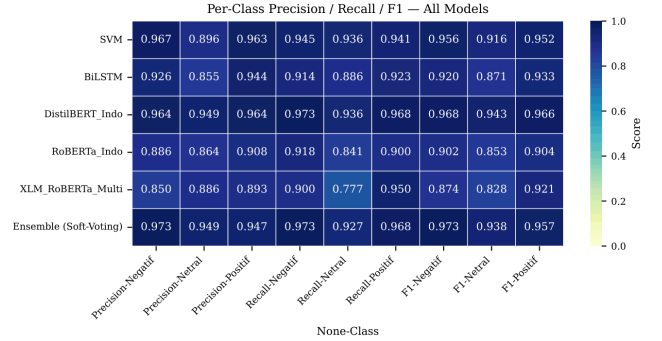


Fig. 2. Per-class precision, recall, and F1-score for the Negatif, Netral, and Positif sentiment categories across all six models.

C. Classification Robustness and Sector Targeting

Although the ensemble narrowly trails DistilBERT_Indo in aggregate MCC, its robustness is strongly validated through its per-class distribution, summarized in Fig. 2. Fig. 3. compares the confusion matrices of all six models. The ensemble achieves a 97.3% per-class recall on the *Negatif* (Negative) category and a 96.8% recall on the *Positif* (Positive) category, figures nearly identical to its strongest individual constituent, DistilBERT_Indo (97.3% and 96.8%, respectively), confirming that the soft-voting mechanism preserves the discriminative strength of its best-performing member on these two classes rather than diluting it.

The *Netral* (Neutral) class remains the most difficult to classify across every model in the matrix, with per-class recall ranging from 77.7% (XLM_RoBERTa_Multi) to 93.6% (SVM and DistilBERT_Indo). The ensemble achieves a 92.7% recall on this class, trailing DistilBERT_Indo by under one percentage point. This consistent *Netral*-class difficulty, despite the corpus's balanced class distribution described in Section III.A, suggests that distinguishing genuinely neutral financial reporting from mildly positive or negative framing is an inherently harder discrimination problem than separating clearly polarized sentiment, a limitation shared across all six

architectures rather than isolated to any single model. This tight diagonal alignment on the *Negatif* (Negative) and *Positif* (Positive) classes ensures that the subsequent Aspect-Based Information Extraction layer receives highly accurate sentiment tags before mapping them to specific economic sectors (e.g., Finance, Energy, Technology) in the live application.

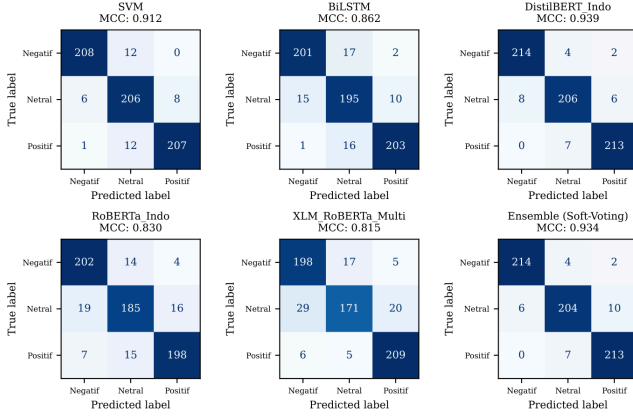


Fig. 3. Confusion matrices for all five constituent models and the soft-voting ensemble on the test set, with corresponding MCC reported per model.

D. Deployment Architecture and End-to-End Latency

To fulfill the requirements for real-world application, the theoretical accuracy of the ensemble was evaluated within a live, decoupled cloud architecture. The system’s frontend user interface was developed using React and deployed via Vercel, while the inference engine and parallel model matrix were hosted independently on Hugging Face Spaces.

Crucially, unlike static academic benchmarks that evaluate pre-loaded text files, the deployed pipeline integrates a live news aggregation module. When a user queries a ticker, the deployed system leverages the requests library to query the Google News RSS Feed API. The XML response is dynamically parsed using `xml.etree.ElementTree` to aggregate the latest article titles and metadata in real-time before routing them directly into the Hugging Face parallel evaluation matrix.

Testing on this production architecture revealed an average end-to-end processing time of 10 to 20 seconds per query sequence. While the isolated tensor execution (model inference) operates rapidly, this expanded 20-second window reflects the unavoidable network I/O overhead required for live data fetching, API routing between Vercel and Hugging Face, and the resource constraints of cloud-hosted endpoints. Although this exceeds the initial theoretical target of 2.5 seconds, it represents a highly successful engineering trade-off. For the intended use case of macro-economic sentiment scanning and retail

investment analysis, where data freshness is prioritized over millisecond-level high-frequency trading speeds, this latency is highly viable and demonstrates the successful deployment of a full-stack, real-time NLP application.

E. Discriminative Performance and Calibration

Beyond point-estimate classification metrics, Section III.F established two additional audit criteria: discriminative separability and probabilistic calibration. Fig. 4. presents one-vs-rest ROC curves across all three sentiment classes for each model, illustrating the threshold-independent separability that complements the MCC results reported in Section IV.A. Notably, even the weakest individual model by MCC, RoBERTa_Indo (0.830), achieves an AUC of 0.978 on the *Positif* class, indicating that its lower MCC stems primarily from a calibration or decision-threshold issue rather than a representational limitation.

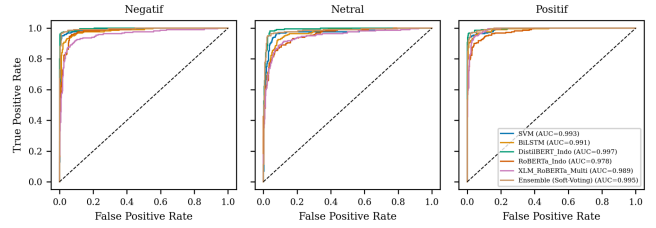


Fig. 4. One-vs-rest ROC curves per sentiment class for all six models, with AUC reported per model in the legend.

Calibration quality was audited against the Log-Loss ceiling of 0.40 specified in Section III.F. As shown in Figure 5, every constituent model satisfied this threshold, with the ensemble achieving the lowest Log-Loss of the matrix (0.175), ahead of DistilBERT_Indo (0.180) despite trailing it slightly on MCC. This indicates that while the ensemble does not produce the single most accurate prediction set, its averaged probability outputs are the best-calibrated in the matrix, a meaningful property for downstream risk-sensitive applications where confidence scores, not just hard labels, inform decision-making.

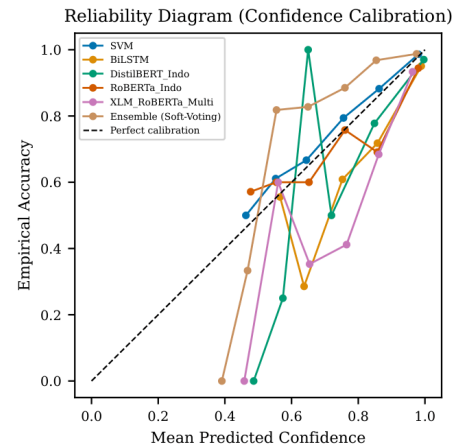


Fig. 5. Reliability diagram comparing mean predicted confidence to empirical accuracy across confidence bins, illustrating calibration quality for each model.

V. CONCLUSION

This study successfully engineered and deployed a real-time, consensus-based NLP pipeline designed to extract sentiment and sector-level impacts from Indonesian financial news. By decoupling a React-based frontend from a parallel Hugging Face inference engine, the project successfully validated a live news aggregation and sentiment architecture suitable for macro-economic scanning.

However, the empirical results nuance the prevailing assumption that deep learning and transformer architectures inherently supersede traditional statistical models, while also revealing that ensembling does not automatically outperform its best constituent. The domain-pretrained transformer, DistilBERT_Indo, achieved the strongest individual performance (MCC 0.939), narrowly ahead of the uniformly-weighted soft-voting ensemble (MCC 0.934) and the TF-IDF-based SVM baseline (MCC 0.912).

This result highlights a critical architectural insight: properly preprocessed, language-specific pretraining allows a transformer to match or exceed both a strong statistical baseline and an ensemble of more diverse but individually weaker architectures, even on a constrained fine-tuning corpus of roughly 3,300 articles. At the same time, the fact that the equally-weighted ensemble could not surpass its strongest constituent demonstrates a key limitation of uniform soft-voting, weaker models are given the same influence as the top performer, diluting the consensus output. Notably, the ensemble nonetheless achieved the lowest Log-Loss across the matrix (0.175), indicating its averaged probability outputs remain the best-calibrated despite the MCC gap, a property of particular value for risk-sensitive financial deployment.

Future work should explore whether expanding the training corpus beyond 3,300 articles narrows the gap between the weaker multilingual and sentiment-pretrained checkpoints (RoBERTa_Indo, XLM_RoBERTa_Multi) and the Indonesia-pretrained DistilBERT_Indo, which already achieves strong performance at the current corpus size. Additionally, future iterations of the consensus engine should explore dynamically adaptive weighting algorithms that automatically monitor validation entropy and sever non-converging networks during live inference. Given that the current uniformly-weighted ensemble narrowly trails its single strongest constituent (DistilBERT_Indo) by only 0.005 MCC, implementing such adaptive weighting is a particularly promising and low-risk direction for closing this gap in future iterations. Ultimately, this research proves that while multi-model cloud pipelines are highly viable for real-time market intelligence, careful alignment between dataset size, lexical density, and model complexity is paramount.

Beyond architectural advancements, this study underscored the critical necessity of stringent pipeline verification. During the initial experimental phase, the

parallel matrix yielded anomalous predictions that misaligned with theoretical expectations. A comprehensive codebase audit was initiated, revealing the project's most significant turning point: the text preprocessing block had been inadvertently bypassed during the execution sequence. Resolving this missing pipeline step drastically stabilized the probabilistic outputs across all models. Ultimately, this research proves that while multi-model cloud pipelines are highly viable for real-time market intelligence, integrating periodic code audits is just as paramount as the mathematical alignment between dataset size, lexical density, and model complexity.

REFERENCES

- [1] S. Shahapur *et al.*, "Discovering untapped potential in finance markets using NLP-driven sentiment analysis," *Indian Journal of Science and Technology*, 2024. [Online]. Available: <https://indjst.org/articles/discovering-untapped-potential-in-finance-markets-using-nlp-driven-sentiment-analysis>
- [2] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," *arXiv preprint arXiv:1908.10063*, 2019.
- [3] K. Mishev *et al.*, "Evaluation of sentiment analysis in finance: From lexicons to transformers," *IEEE Access*, 2020, doi: 10.1109/ACCESS.2020.3009626.
- [4] I. Almalis *et al.*, "Sector-level sentiment analysis with deep learning," *Knowledge-Based Systems*, 2022, doi: 10.1016/j.knosys.2022.109954.
- [5] G. Piao and J. G. Breslin, "Financial aspect and sentiment predictions with deep neural networks: An ensemble approach," in *Proc. of the ACM Web Conference*, 2018. [Online]. Available: <https://dl.acm.org/doi/10.1145/3184558.3191829>
- [6] K. Alissa and O. Alzoubi, "Financial sentiment analysis based on transformers and majority voting," in *Proc. of the IEEE/ACS 19th International Conference on Computer Systems and Applications (AICCSA)*, 2022, doi: 10.1109/AICCSA56895.2022.10017941.
- [7] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in *Proc. of the European Conference on Machine Learning (ECML)*, 1998. [Online]. Available: https://www.cs.cornell.edu/~tj/publications/joachims_98a.pdf
- [8] P. Zhou *et al.*, "Attention-based bidirectional long short-term memory networks for relation classification," in *Proc. of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016. [Online]. Available: <https://aclanthology.org/P16-2034.pdf>
- [9] P. He *et al.*, "DeBERTa: Decoding-enhanced BERT with disentangled attention," *arXiv preprint arXiv:2006.03654*, 2020.
- [10] V. Sanh *et al.*, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.
- [11] J. Devlin *et al.*, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019. [Online]. Available: <https://aclanthology.org/N19-1423.pdf>
- [12] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

- [13] T. Loughran and B. McDonald, "When is a liability not a liability? Textual analysis, dictionaries, and 10-Ks," *The Journal of Finance*, 2011, doi: 10.1111/j.1540-6261.2010.01625.x.
- [14] J. Bollen, H. Mao, and X. Zeng, "Twitter mood predicts the stock market," *Journal of Computational Science*, 2011, doi: 10.1016/j.jocs.2010.12.007.
- [15] F. Koto *et al.*, "IndoLEM and IndoBERT: A benchmark dataset and pre-trained language model for Indonesian NLP," in *Proc. of the 28th International Conference on Computational Linguistics (COLING)*, 2020. [Online]. Available: <https://aclanthology.org/2020.coling-main.66.pdf>

APPENDIX

Name	Contribution
Nathanael Setiorahardjo	• Manuscript and research documentation • Baseline machine learning model prototyping. • Frontend web application co-development. • Auditing and correcting missing Preprocessing block
Gavinn Aloys	• Core machine learning engineering and optimization • Experimental computation (5-model parallel matrix) • System deployment and cloud integration (MLOps)
Kelvin Leandi	• Data engineering and live web scraping • Dataset construction and text preprocessing heuristics • Frontend web application co-development

A. Pipeline Initialization and Data Preprocessing

To ensure absolute reproducibility across all computational runs and hardware architectures, the pipeline enforces a strict deterministic initialization across all random number generators. The unstructured textual data is subsequently normalized utilizing a custom regex-based preprocessing function that standardizes the casing, eradicates HTML/URL noise, and explicitly preserves market-sensitive financial indicators.

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)
set_seed(SEED) # locks HuggingFace Trainer init/sampler/dropout RNGs
```

```

df = pd.read_csv('scraped_dataset_NLP.csv')
df = df.dropna(subset=['Judul', 'Sentimen'])

X_raw = df['Judul'].astype(str)
y_text = df['Sentimen']

def clean_financial_headline(text):
    text = text.lower()
    text = re.sub(r'<[^>]+>', ' ', text)
    text = re.sub(r'http\S+|www\S+', ' ', text)
    # keep letters, numbers, spaces, and financial symbols incl. direction arrows
    text = re.sub(r'^\w\s\$\%\+|-\.|u2191\u2193', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

print("Cleaning headlines: lowercasing, stripping noise, preserving $ % + - . , and direction arrows...")
X = X_raw.apply(clean_financial_headline)

label_mapping = {'Negatif': 0, 'Netral': 1, 'Positif': 2}
target_names = ['Negatif', 'Netral', 'Positif']
y = y_text.map(label_mapping)
num_labels = 3

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED, stratify=y
)
print(f"Train: {len(X_train)} | Test: {len(X_test)}")

pred_probs = {}
model_metrics = []

```

B. Baseline Architectures and Transformer Fine-Tuning

The experimental matrix implements computational baselines alongside advanced contextual networks. The statistical baseline utilizes a Support Vector Machine (SVM) coupled with a bounded TF-IDF vectorizer and SMOTE.

For the transformer networks, a RandomOverSampler is applied to flatten the class distribution prior to tokenization. The fine-tuning process utilizes the Hugging Face Trainer API with mixed precision (FP16).

```
# =====
# 5. TRAIN & SAVE: SVM (with SMOTE)
# =====
print("[1/5] Training SVM (TF-IDF + SMOTE)...")
vectorizer = TfidfVectorizer(lowercase=True, max_features=5000)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

smote = SMOTE(random_state=SEED)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_vec, y_train)

svm_model = SVC(kernel='linear', probability=True, random_state=SEED)
svm_model.fit(X_train_resampled, y_train_resampled)

joblib.dump(svm_model, os.path.join(SAVE_DIR, "svm_model.pkl"))
joblib.dump(vectorizer, os.path.join(SAVE_DIR, "tfidf_vectorizer.pkl"))

pred_probs['SVM'] = svm_model.predict_proba(X_test_vec)
svm_pred = np.argmax(pred_probs['SVM'], axis=1)

model_metrics.append({
    'Model': 'SVM', 'Accuracy': accuracy_score(y_test, svm_pred),
    'F1-Score': f1_score(y_test, svm_pred, average='weighted'),
    'MCC': matthews_corrcoef(y_test, svm_pred), 'Log-Loss': log_loss(y_test, pred_probs['SVM'])
})
print("SVM done.")
```

```
# =====
# 7. TRAIN & SAVE: TRANSFORMERS (DistilBERT_Indo, RoBERTa_Indo, XLM_RoBERTa_Multi)
# =====
ros = RandomOverSampler(random_state=SEED)
X_train_ros, y_train_ros = ros.fit_resample(np.array(X_train).reshape(-1, 1), y_train)
X_train_ros = X_train_ros.flatten()

train_df = pd.DataFrame({'text': X_train_ros, 'label': y_train_ros})
test_df = pd.DataFrame({'text': X_test, 'label': y_test})
train_dataset = Dataset.from_pandas(train_df, preserve_index=False)
test_dataset = Dataset.from_pandas(test_df, preserve_index=False)

transformer_models = {
    "DistilBERT_Indo": "indobenchmark/indobert-base-p1",
    "RoBERTa_Indo": "w11wo/indonesian-roberta-base-sentiment-classifier",
    "XLM_RoBERTa_Multi": "xlm-roberta-base"
}

for i, (name, checkpoint) in enumerate(transformer_models.items(), start=3):
    print(f"[{i}/5] Training {name}...")

    tokenizer = AutoTokenizer.from_pretrained(checkpoint)
    model = AutoModelForSequenceClassification.from_pretrained(
        checkpoint, num_labels=num_labels, ignore_mismatched_sizes=True
    )

    def tokenize_function(examples):
        return tokenizer(examples["text"], padding="max_length", truncation=True, max_length=64)

    tokenized_train = train_dataset.map(tokenize_function, batched=True)
    tokenized_test = test_dataset.map(tokenize_function, batched=True)
```

```

training_args = TrainingArguments(
    output_dir=f"./temp_{name}",
    eval_strategy="no",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    num_train_epochs=3,
    logging_steps=50,
    report_to="none",
    fp16=True,
    save_strategy="no",
    seed=SEED,
    data_seed=SEED,
)

trainer = Trainer(model=model, args=training_args, train_dataset=tokenized_train)
trainer.train()

model_save_path = os.path.join(SAVE_DIR, name)
trainer.save_model(model_save_path)
tokenizer.save_pretrained(model_save_path)
print(f"{name} saved to {model_save_path}")

raw_predictions = trainer.predict(tokenized_test)
logits = raw_predictions.predictions
probabilities = torch.nn.functional.softmax(torch.tensor(logits), dim=-1).numpy()
pred_probs[name] = probabilities

model_pred = np.argmax(probabilities, axis=1)
model_metrics.append({
    'Model': name, 'Accuracy': accuracy_score(y_test, model_pred),
    'F1-Score': f1_score(y_test, model_pred, average='weighted'),
    'MCC': matthews_corrcoef(y_test, model_pred), 'Log-Loss': log_loss(y_test, probabilities)
})

del model, trainer, tokenizer
gc.collect()
if torch.cuda.is_available():
    torch.cuda.empty_cache()

print("All transformers done.")

```

C. Consensus Aggregation and Extended Evaluation Suite

Following the independent inference phase, the system aggregates the probability vectors utilizing a uniform soft-voting consensus mechanism, matching the mathematical formulation established in the study. To ascertain whether the performance disparities between the top models are statistically significant, pairwise McNemar's tests are executed on the final test predictions.

```

# =====
# 8. SOFT-VOTING ENSEMBLE (equal-weight average – matches paper Eq. 1)
# =====
ensemble_probs = sum(pred_probs.values()) / len(pred_probs)
ensemble_pred = np.argmax(ensemble_probs, axis=1)

model_metrics.append({
    'Model': 'Ensemble (Soft-Voting)', 'Accuracy': accuracy_score(y_test, ensemble_pred),
    'F1-Score': f1_score(y_test, ensemble_pred, average='weighted'),
    'MCC': matthews_corrcoef(y_test, ensemble_pred), 'Log-Loss': log_loss(y_test, ensemble_probs)
})

comparison_df = pd.DataFrame(model_metrics)
print("\n=== ALL MODELS – COMPARISON ===")
print(comparison_df.to_string(index=False))

print("\n=== CLASSIFICATION REPORT: ENSEMBLE ===")
print(classification_report(y_test, ensemble_pred, target_names=target_names))

```

D. Model Optimization and ONNX Conversion

To fulfill the sub-second latency requirements for real-world deployment, the constituent models were exported to the Open Neural Network Exchange (ONNX) format. The transformer models were converted directly utilizing the `optimum.onnxruntime` library.

```

# ONNX Conversion Pipeline for Transformers[cite: 2]
from optimum.onnxruntime import ORTModelForSequenceClassification

tokenizer = AutoTokenizer.from_pretrained(f"./{source_folder}")
model = ORTModelForSequenceClassification.from_pretrained(f"
    ./{source_folder}", export=True)

# Export to lightweight deployment folder[cite: 2]
tokenizer.save_pretrained(f"./{target_folder}")
model.save_pretrained(f"./{target_folder}")

```

For the BiLSTM architecture, a direct Keras object conversion was bypassed. Instead, the raw mathematical operations were traced via a TensorFlow function signature to generate a pure math graph, ensuring high-efficiency compatibility with the ONNX opset.

```
# BiLSTM ONNX Conversion via Pure Math Graph Trace[cite: 3]
import tf2onnx
import onnx

@tf.function(input_signature=[tf.TensorSpec([None, 50], tf.int32, name="input_ids")])
def trace_model(inputs):
    return model(inputs)

onnx_model, _ = tf2onnx.convert.from_function(
    trace_model,
    input_signature=[tf.TensorSpec([None, 50], tf.int32, name="input_ids")]
,
    opset=13
)
onnx.save(onnx_model, onnx_output_path)
```

E. Frontend Web Application and Financial Widget Integration

The real-time user interface was developed using React.js with a component-based architecture. To monitor live market movements and correlate them with the sentiment analysis outputs, the frontend integrates external financial charting tools dynamically, ensuring a seamless visualization of economic sector impacts.

```
// Frontend Core Dependencies and Isolated Widget Component
import React, { useState, useEffect, useRef } from "react";
import { Search, Activity, BarChart2, Database, Users } from "lucide-react";

// --- ISOLATED TRADINGVIEW COMPONENT ---
const TradingViewWidget = ({ keyword }) => {
    const containerRef = useRef(null);

    useEffect(() => {
        if (!containerRef.current) return;
        containerRef.current.innerHTML = ""; // Clear previous instances

        const script = document.createElement("script");
        script.src = "https://s3.tradingview.com/external-embedding/embed-widget-advanced-chart.js";
        script.type = "text/javascript";
        script.async = true;
        script.innerHTML = JSON.stringify({
            autosize: true,
            symbol: keyword ? `IDX:${keyword}` : "IDX:COMPOSITE",
            interval: "D",
            timezone: "Etc/UTC",
            theme: "light",
            style: "1",
            locale: "en",
            enable_publishing: false,
            hide_top_toolbar: true,
            hide_legend: true,
            save_image: false,
            calendar: false,
            support_host: "https://www.tradingview.com"
        });
        containerRef.current.appendChild(script);
    }, [keyword]);

    return (
        <div className="w-full h-[300px] md:h-[400px] overflow-hidden rounded
        -xl border border-gray-100 shadow-sm relative z-0">
            <div className="tradingview-widget-container__widget w-full h-full"
            ref={containerRef}></div>
        </div>
    );
};
```

The main application layout employs a state-driven navigation system to switch between the sentiment terminal, methodology breakdown, and team information panels.

```
// Application View State Management
const [currentView, setCurrentView] = useState("app");

// Navigation Panel Render
<div className="flex items-center justify-around h-16 px-2">
  <button
    onClick={() => setCurrentView("app")}
    className={`flex flex-col items-center justify-center w-full h-full gap
-1 transition-colors ${currentView === "app" ? "text-orange-500" : "text
-gray-400 hover:text-gray-900"}`>
    <BarChart2 size={20} />
    <span className="text-[10px] font-bold tracking-wide">Terminal</span>
  </button>
  <button
    onClick={() => setCurrentView("science")}
    className={`flex flex-col items-center justify-center w-full h-full gap
-1 transition-colors ${currentView === "science" ? "text-orange-500" :
"text-gray-400 hover:text-gray-900"}`>
    <Database size={20} />
    <span className="text-[10px] font-bold tracking-wide">Methodology</span>
  </button>
</div>
```

F. Application Screenshots

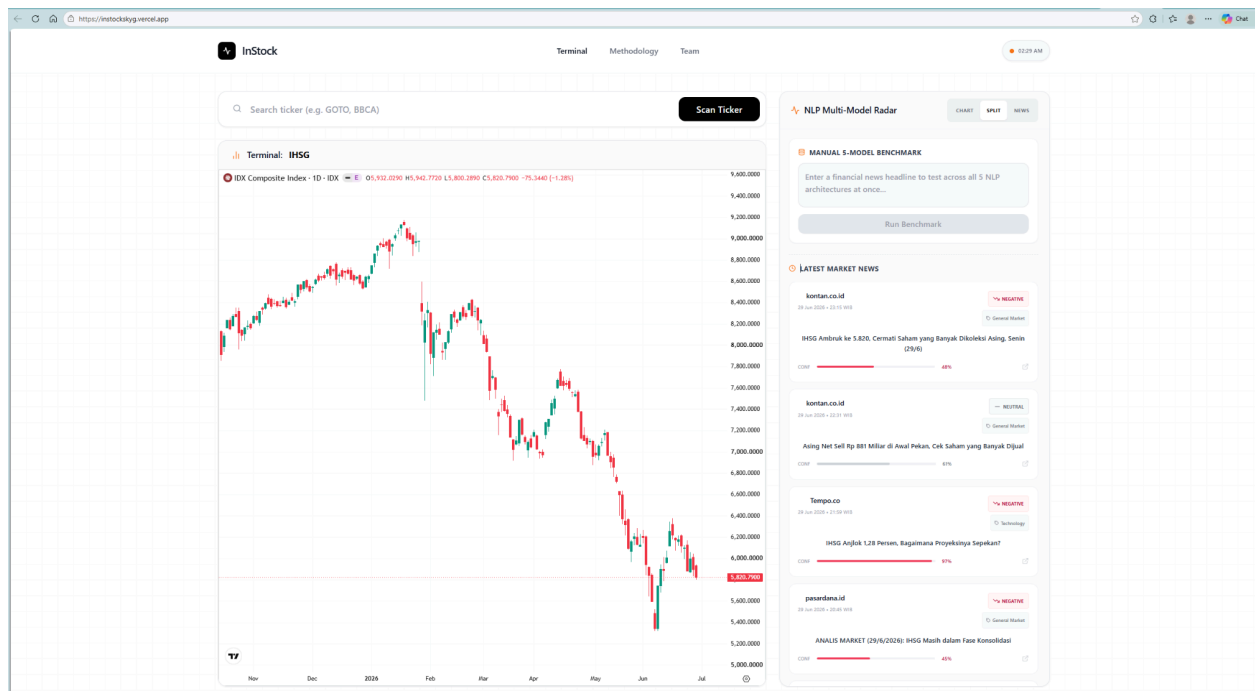


Fig. 1. The main dashboard of the InStock application, displaying real-time financial sentiment tracking and the integrated TradingView chart for specific stock tickers.

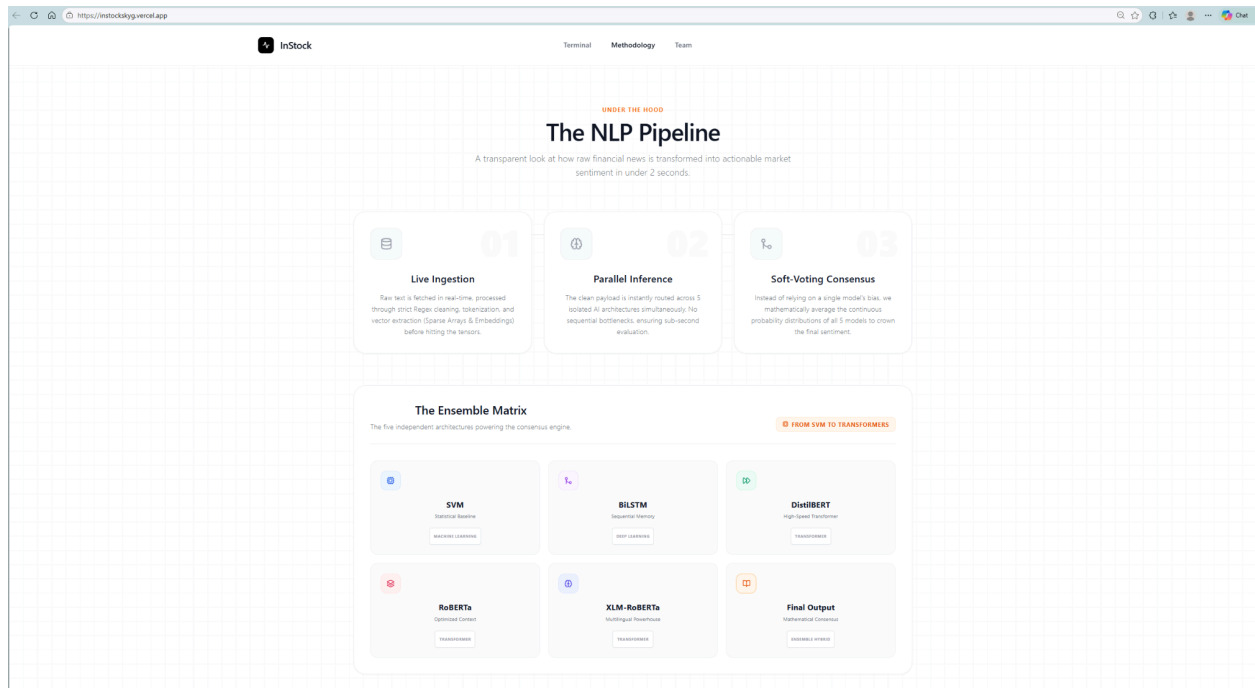


Fig. 2. The methodology interface illustrates the architecture of the consensus-based ensemble model and confidence score metrics.

G. Backend Inference Engine and Live API Integration

The backend inference engine was developed utilizing the FastAPI framework to ensure high-performance, asynchronous request handling for the parallel model matrix. To optimize memory management and prevent out-of-memory (OOM) errors during production deployment, the five constituent models (SVM, BiLSTM, DistilBERT, RoBERTa, and XLM-RoBERTa) are instantiated into a centralized state dictionary during the application's lifespan event.

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    print("🚀 Booting up 5-Model Custom ONNX Ensemble...")

    try:
        # 1. Load Local SVM
        ml_models["svm"] = joblib.load(f"{MODEL_DIR}/svm_model.pkl")
        ml_models["vectorizer"] = joblib.load(f"{MODEL_DIR}/tfidf_vectorizer.pkl")
        print(f"    -> Sklearn version : {sklearn.__version__}")
        print(f"    -> Vectorizer type : {type(ml_models['vectorizer'])}")
        try:
            print(f"    -> IDF fitted      : {ml_models['vectorizer'].idf_ is not None}")
            print(f"    -> Vocab size      : {len(ml_models['vectorizer'].vocabulary_)}")
        except AttributeError as e:
            print(f"    -> ✗ NOT FITTED AT LOAD TIME: {e}")
        print(f"    -> ✅ SVM Loaded")

        # 2. Load Local BiLSTM
        ml_models["bilstm_session"] = ort.InferenceSession(f"{MODEL_DIR}/bilstm_model.onnx")
        with open(f"{MODEL_DIR}/bilstm_tokenizer.pkl", "rb") as f:
            ml_models["bilstm_tokenizer"] = pickle.load(f)
        print(f"    -> ✅ BiLSTM Loaded")

        # 3, 4, 5. Download and Load YOUR Custom Transformers
        print(f"    -> 📦 Downloading Custom Transformer Weights from HF Hub...")
        custom_weights_path = snapshot_download(repo_id=CUSTOM_MODEL_REPO)

        transformers = ["distilbert", "roberta", "xlm_roberta"]
        for t_name in transformers:
            t_path = os.path.join(custom_weights_path, f"{t_name}_onnx")
            ml_models[f"{t_name}_tok"] = AutoTokenizer.from_pretrained(t_path)
            ml_models[f"{t_name}_session"] = ort.InferenceSession(os.path.join(t_path, "model.onnx"))
            print(f"    -> ✅ Custom {t_name.replace('_', ' ').title()} Loaded")

        ml_models["is_loaded"] = True
        print(f"🟢 5-MODEL CUSTOM ENSEMBLE FULLY OPERATIONAL!")

    except Exception as e:
        print(f"⚠️ Load Failed. Falling back to Lexical Engine. Error: {e}")
        ml_models["is_loaded"] = False

    yield
    ml_models.clear()
    gc.collect()

app = FastAPI(title="InStock NLP Engine", lifespan=lifespan)

```

To aggregate the predictions from the parallel model matrix, the consensus engine applies a Uniform Soft-Voting strategy. Rather than executing a hard-voting majority rule, the algorithm calculates the arithmetic mean of the continuous probability distributions generated by all five models. By giving each constituent an equal unweighted influence, the system stabilizes the variance of individual predictions to produce a single, unified consensus score.

```
# Uniform Soft-Voting Consensus Probability Aggregation
def analyze_sentiment(text: str):
    if not ml_models.get("is_loaded"):
        return "Neutral", 0.50
    try:
        # Extract probability distributions from all 5 models
        vec_text = ml_models["vectorizer"].transform([text])
        prob_svm = ml_models["svm"].predict_proba(vec_text)[0]

        seq = ml_models["bilstm_tokenizer"].texts_to_sequences([text])
        pad_seq = pad_sequences_np(seq, maxlen=50)
        prob_bilstm = ml_models["bilstm_session"].run(None, {ml_models["bilstm_session"].get_inputs()[0].name: pad_seq})[0][0]

        prob_distil = get_transformer_probs(text, "distilbert")
        prob_roberta = get_transformer_probs(text, "roberta")
        prob_xlm = get_transformer_probs(text, "xlm_roberta")

        # 5-Model Soft-Voting Core Math (Arithmetic Mean)
        avg_probs = (prob_svm + prob_bilstm + prob_distil + prob_roberta + prob_xlm) / 5.0

        labels = ["Negative", "Neutral", "Positive"]
        max_index = np.argmax(avg_probs)
        return labels[max_index], float(avg_probs[max_index])
    except Exception as e:
        print(f"Prediction Error: {e}")
        return "Neutral", 0.50
```

For aspect-level information extraction, the backend bypasses computationally expensive dependency parsing algorithms in favor of a rapid Lexical Rule-Based Engine. Unstructured payloads are normalized and cross-referenced against a predefined array of industry-specific corporate entities, guaranteeing sub-second routing to the correct macro-economic domain.

```
def extract_economic_sector(text: str):
    text_lower = text.lower()
    sectors = {
        "Finance": ["bank", "suku bunga", "kredit", "bca", "bri", "mandiri", "bni", "ojk", "bi", "finansial"],
        "Energy & Mining": ["batu bara", "migas", "tambang", "nikel", "adaro", "ptba", "medc", "energi", "emas"],
        "Consumer Goods": ["ritel", "makanan", "indofood", "unilever", "minuman", "fmcg", "pasar", "belanja"],
        "Technology": ["goto", "digital", "startup", "ecommerce", "aplikasi", "telkom", "ai", "tech"],
        "Infrastructure": ["jalan tol", "wika", "pembangunan", "konstruksi", "semen", "properti", "perumahan"]
    }
    for sector, keywords in sectors.items():
        if any(keyword in text_lower for keyword in keywords):
            return sector
    return "General Market"
```

To ensure the system analyzes current market conditions, a decoupled live news aggregator was integrated into the primary API endpoint. The script utilizes an asynchronous request mechanism to parse XML responses from the Google News RSS Feed, automatically injecting live localized financial headlines directly into the parallel inference matrix.

```

def fetch_live_news(ticker: str):
    encoded_query = urllib.parse.quote(f'{ticker} saham OR bursa OR keuangan')
    url = f'https://news.google.com/rss/search?q={encoded_query}&hl=id&gl=ID&ceid=ID:id'
    raw_articles = []

    try:
        response = requests.get(url, headers={'User-Agent': 'Mozilla/5.0'}, timeout=5)
        root = ET.fromstring(response.content)
        for item in root.findall('./channel/item'):
            full_title = item.find('title').text
            title, source = full_title.rsplit(" - ", 1) if " - " in full_title else (full_title, "Berita Finansial")
            try:
                time_tuple = email.utils.parsedate_tz(item.find('pubDate').text)
                dt_wib = datetime.fromtimestamp(email.utils.mktime_tz(time_tuple), timezone.utc) + timedelta(hours=7)
                raw_articles.append({'id': str(time.time() + len(raw_articles)), "title": title, "source": source, "dt_obj": dt_wib, "url": item.find('link').text})
            except: continue

        raw_articles.sort(key=lambda x: x["dt_obj"], reverse=True)
        final_articles = []
        bulan_indo = {1:"Jan", 2:"Feb", 3:"Mar", 4:"Apr", 5:"Mei", 6:"Jun", 7:"Jul", 8:"Agt", 9:"Sep", 10:"Okt", 11:"Nov", 12:"Des"}
        for article in raw_articles[:10]:
            d = article.pop("dt_obj")
            article["time"] = f"{d.day:02d} {bulan_indo[d.month]} {d.year} • {d.hour:02d}:{d.minute:02d} WIB"
            final_articles.append(article)
        return final_articles
    except: return []

@app.post("/api/scan")
async def scan_market(request: SearchRequest):
    start_time = time.time()
    real_news_data = fetch_live_news(request.ticker)

    for article in real_news_data:
        sentiment, score = analyze_sentiment(article["title"])
        article["sentiment"] = sentiment
        article["score"] = score
        article["sector"] = extract_economic_sector(article["title"])

    return {
        "status": "success",
        "latency": f"{round(time.time() - start_time, 2)}s",
        "data": real_news_data
    }

```